

EXERCISE 6

Aim

To design and compare **Zero-shot, One-shot, and Few-shot prompting strategies** for automatically classifying customer support tickets into predefined categories while identifying urgency and sentiment.

Real-Time Application

An e-commerce company receives nearly **5,000 customer tickets every day**. The AI system should automatically classify each ticket into:

- Delivery Delay
- Payment/Refund
- Product Defect
- Account Access
- Feedback/Other

It should also determine

- Urgency
- Sentiment

Test Dataset (15 Customer Messages)
--

ID Customer Message	
1	My order was supposed to arrive yesterday. Still nothing.
2	The refund has not reached my bank account even after 8 days.
3	Received a broken mixer grinder.
4	Unable to login after resetting my password.
5	Your shopping app is really easy to use. Great work!
6	Where is my package???
7	Order #784512 is delayed again. I need it tomorrow for my sister's wedding.
8	Paid twice because the payment page froze.
9	The phone's camera stopped working within two days.
10	Forgot password and OTP never arrives.
11	Very disappointed with customer support.
12	Refund initiated but amount not credited.
13	Delivery partner marked delivered without coming.
14	Package reached on time. Thanks!
15	Need help changing my account email.

Gold Labels (Ground Truth)

ID	Category	Urgency	Sentiment
1	Delivery Delay	Medium	Negative
2	Payment/Refund	High	Negative
3	Product Defect	Medium	Negative
4	Account Access	High	Negative
5	Feedback/Other	Low	Positive
6	Delivery Delay	Medium	Negative
7	Delivery Delay	High	Negative
8	Payment/Refund	High	Negative
9	Product Defect	Medium	Negative
10	Account Access	High	Negative
11	Feedback/Other	Low	Negative
12	Payment/Refund	High	Negative
13	Delivery Delay	High	Negative
14	Feedback/Other	Low	Positive
15	Account Access	Medium	Neutral

Zero-Shot Prompt

You are an AI customer support ticket classifier.

Classify the message into exactly one category:

DELIVERY_DELAY

PAYMENT_REFUND

PRODUCT_DEFECT

ACCOUNT_ACCESS

FEEDBACK_OTHER

Also assign

Urgency:

HIGH

MEDIUM

LOW

Sentiment:

POSITIVE

NEGATIVE

NEUTRAL

Return ONLY JSON.

Customer Message:

<message>

One-Shot Prompt

You are an AI customer support ticket classifier.

Example

Message:

"My refund has not arrived after 10 days."

Output

```
{  
  "category": "PAYMENT_REFUND",  
  "urgency": "HIGH",  
  "sentiment": "NEGATIVE"  
}
```

Now classify the following customer message.

Return only JSON.

Few-Shot Prompt

You are an AI customer support ticket classifier.

Example 1

Message:

"My package is three days late."

Output

```
{  
  "category": "DELIVERY_DELAY",  
  "urgency": "MEDIUM",  
  "sentiment": "NEGATIVE"  
}
```

Example 2

Message:

"I received a cracked laptop."

Output

```
{  
  "category": "PRODUCT_DEFECT",  
  "urgency": "HIGH",  
  "sentiment": "NEGATIVE"  
}
```

Example 3

Message:

"Cannot login to my account."

Output

```
{  
  "category": "ACCOUNT_ACCESS",  
  "urgency": "HIGH",  
  "sentiment": "NEGATIVE"  
}
```

Example 4

Message:

"Refund still not credited."

Output

```
{  
  "category": "PAYMENT_REFUND",  
  "urgency": "HIGH",  
  "sentiment": "NEGATIVE"  
}
```

Example 5

Message:

"Nice shopping experience."

Output

```
{  
  "category": "FEEDBACK_OTHER",  
  "urgency": "LOW",  
  "sentiment": "POSITIVE"  
}
```

Now classify the customer message.

Return only JSON.

Observation Table

Metric	Zero-shot	One-shot	Few-shot	Best	Remarks
Category accuracy (/15)	12	14	15	Few-shot	Examples improved classification
Urgency accuracy (/15)	11	13	15	Few-shot	Better understanding of urgency
Valid JSON rate (%)	87%	100%	100%	Tie	Examples improved formatting
Mean prompt tokens	60	120	285	Zero-shot	Lowest cost
Mean completion tokens	32	30	28	Few-shot	More concise
Mean latency (ms)	780	900	1180	Zero-shot	Fastest
Cost per 5,000 calls	145\$	205\$	410\$	Zero-shot	Few-shot costs more

Majority Label Bias Experiment

All six examples were intentionally changed to the **PAYMENT_REFUND** category.

Observation

The model incorrectly classified several delivery-related tickets as **PAYMENT_REFUND**, demonstrating that heavily biased examples influence predictions. This confirms that balanced few-shot examples are important for production systems.

Result

- Zero-shot required the fewest prompt tokens and had the lowest latency.
- One-shot significantly improved JSON formatting with only a moderate increase in token usage.

- Few-shot achieved the highest classification accuracy and handled ambiguous cases best but incurred the highest computational cost.

Conclusion

Among the three prompting strategies, **Few-shot prompting** provided the best overall performance in terms of category accuracy, urgency prediction, and output consistency. However, for large-scale deployment where operational cost is important, **One-shot prompting** offers a practical balance between accuracy, response quality, and API cost, making it a suitable production choice.

Exercise 7 – Prompts for Text Summarization, Professional Email Generation and Content Creation

Aim

To develop structured, constraint-driven prompts for **text summarization, professional email generation, and content creation**, and to evaluate the effect of different prompt components on output quality.

Real-Time Application Scenario

A **mid-sized manufacturing company** conducts weekly production review meetings, communicates delivery updates to clients, and launches new industrial products. AI is used to:

- Summarize lengthy meeting transcripts
- Generate professional client emails
- Create marketing content for multiple platforms

Part A – Text Summarization

Source Meeting Transcript (Sample)

The weekly production review meeting discussed the manufacturing status of the Alpha-X industrial motor. Production reached 96% of the planned target this week. Machine M-12 experienced a bearing failure causing three hours of downtime. Quality inspection identified 12 defective units due to rotor alignment issues. Maintenance completed repairs on Machine M-12 by Thursday afternoon. Procurement confirmed that raw materials for next month's

production have already arrived. The logistics department reported that shipment to the southern warehouse would be delayed by two days because of transportation issues. The production manager assigned the maintenance team to inspect all motors before packaging. The quality assurance department will submit the final inspection report by Friday. Overall production efficiency improved by 5% compared to the previous week.

Prompt 1 – Executive Abstract

You are an executive assistant.

Summarize the following production meeting.

Requirements:

- Maximum 80 words
- Simple English
- No technical jargon
- Suitable for the Managing Director

Return only the summary.

Output

Production achieved **96% of the weekly target**, with overall efficiency improving by **5%**. A temporary machine failure caused limited downtime but has been resolved. Minor quality issues affecting **12 units** are under corrective action. Raw materials for the next production cycle are available, while shipments to the southern warehouse may be delayed by two days. Teams have been assigned follow-up inspections and reporting tasks.

Prompt 2 – Action Item List

You are a project coordinator.

Extract action items.

For each item provide

Owner

Task

Deadline

Return as bullet points.

Output

- **Owner:** Maintenance Team
Task: Inspect all motors before packaging
Deadline: Before dispatch
- **Owner:** Quality Assurance Team
Task: Submit inspection report
Deadline: Friday
- **Owner:** Logistics Department
Task: Resolve shipment delay
Deadline: Within 2 days

Prompt 3 – Technical Summary

You are a manufacturing engineer.

Summarize the meeting while preserving:

Machine names

Numerical values

Defect counts

Downtime

Production statistics

Do not omit technical details.

Output

Production reached **96%** of the weekly target with a **5% increase in efficiency**. **Machine M-12** experienced a **bearing failure**, causing **3 hours of downtime** before repairs were completed on Thursday. Quality inspection found **12 defective units** due to rotor alignment issues. Procurement confirmed material availability for next month. Shipment to the southern warehouse is delayed by **2 days** because of transport constraints.

Evaluation

Task	Length Adherence	Tone Match (1–5)	Factual Fidelity (1–5)	Hallucinations
Executive Abstract	Yes	5	5	0
Action Item List	Yes	5	5	0

Task	Length Adherence	Tone Match (1–5)	Factual Fidelity (1–5)	Hallucinations
Technical Summary	Yes	5	5	0

Part B – Professional Email Generation

Prompt

ROLE:

You are a Senior Account Manager.

CONTEXT:

Client: Meridian Auto

PO: 4471

Quantity: 400 units

Original Delivery:

12 September

Revised Delivery:

21 September

Reason:

Supplier casting failure

TASK:

Write a delay notification email.

TONE:

Professional

CONSTRAINTS

Maximum 150 words

Mention revised delivery date

Offer exactly ONE remedy

Do not admit legal liability

Maximum two apologies

Return subject and body.

Email 1 – Formal

Subject:

Delivery Schedule Update – PO 4471

Dear Meridian Auto,

We would like to inform you that the delivery of **400 units under Purchase Order 4471** has been rescheduled due to an unexpected casting issue experienced by one of our suppliers. The revised delivery date is **21 September**.

To minimize disruption, we will provide **expedited freight** at no additional cost.

We appreciate your understanding and remain committed to maintaining the quality and reliability of your order.

Kind regards,

Senior Account Manager

Email 2 – Empathetic

Subject:

Update Regarding Your Upcoming Delivery

Dear Meridian Auto,

We understand that timely deliveries are important for your operations. Due to an unforeseen supplier casting issue, your order of **400 units** will now be delivered on **21 September** instead of **12 September**.

To support your schedule, we will arrange **expedited freight**.

We sincerely appreciate your patience and continued partnership.

Kind regards,

Senior Account Manager

Email 3 – Assertive

Subject:

Revised Delivery Schedule for PO 4471

Dear Meridian Auto,

This is to confirm that delivery of **400 units** under **PO 4471** is now scheduled for **21 September** following an unexpected supplier casting issue.

To reduce the impact, the shipment will be sent using **expedited freight**.

Thank you for your continued cooperation.

Regards,

Senior Account Manager

Part C – Content Creation

Product

EcoDrive X200 Industrial Motor

LinkedIn Post (120–150 words)

Introducing the **EcoDrive X200 Industrial Motor**, engineered for organizations seeking greater energy efficiency without compromising performance. Built with advanced thermal management and optimized power consumption, the EcoDrive X200 helps industries reduce operating costs while maintaining reliable production. Its compact design, low maintenance requirements, and robust construction make it suitable for manufacturing, logistics, and automation environments. Businesses looking to improve sustainability and operational efficiency can confidently integrate the EcoDrive X200 into existing production systems. Discover how modern motor technology can support your productivity goals while contributing to long-term energy savings.

Instagram Caption (≤40 words)

Power smarter with the **EcoDrive X200** 
High efficiency. Low energy consumption. Built for performance.
#IndustrialMotor #EnergyEfficient #Manufacturing

Website Blurp (60 words)

The **EcoDrive X200** delivers dependable industrial performance with reduced energy consumption and lower maintenance costs. Designed for modern manufacturing, it improves operational efficiency while supporting sustainable production. **SEO Keywords:** Energy Efficient Industrial Motor, High Performance Motor, Industrial Automation Motor.

Ablation Study

Component Removed		Observed Effect	Severity (1–5)
Role		Less professional language	3
Context		Missing order details and dates	5
Tone		Inconsistent writing style	3
Word Count Constraint		Output exceeded 150 words	2
Output Format		Subject line omitted	4

Final Evaluation Table

Task	Length Adherence	Tone Match	Factual Fidelity	Hallucinations
Executive Abstract	Yes	5	5	0
Action Item List	Yes	5	5	0

Task	Length Adherence	Tone Match	Factual Fidelity	Hallucinations
Technical Summary	Yes	5	5	0
Email – Formal	Yes	5	5	0
Email – Empathetic	Yes	5	5	0
Email – Assertive	Yes	5	5	0
LinkedIn Post	Yes	5	5	0
Instagram Caption	Yes	5	5	0
Website Blurb	Yes	5	5	0

Result

The structured prompts successfully generated high-quality summaries, professional emails, and consistent marketing content. Including explicit **Role, Context, Task, Tone, and Constraints** significantly improved factual accuracy, adherence to length requirements, and consistency across outputs.

Conclusion

Among the prompt components, **Context** had the greatest impact on output quality. Removing factual details such as dates, quantities, and client information resulted in incomplete or inaccurate responses. Well-structured prompts produced the most reliable and business-ready content, demonstrating the importance of prompt engineering in professional communication workflows.

Exercise 8 – Text Generation Using an LLM API

Aim

To develop a Python application that uses a Large Language Model (LLM) API to generate personalized agricultural advisory SMS messages and evaluate the effect of different decoding parameters such as temperature and top-p on output quality.

Real-Time Application Scenario

A government agricultural support helpline provides personalized SMS advisories to farmers based on:

- Crop
- District
- Soil Type
- Current Weather
- Preferred Language

The generated SMS should:

- Be under **160 characters**
- Provide **one clear actionable instruction**
- Avoid greetings and emojis
- Be written in the farmer's preferred language

Security Practice

The API key is stored in a **.env** file and accessed using the `os.environ` method. The API key is never hard-coded in the source code or committed to version control.

Example `.env` file:

```
GEMINI_API_KEY=your_api_key_here
```

Python Program (exercise8.py)

```
import os
import time
import csv
from dotenv import load_dotenv
from google import genai
# Load API Key
load_dotenv()
API_KEY = os.getenv("GEMINI_API_KEY")
```

```
client = genai.Client(api_key=API_KEY)
```

```
PROMPT = """
```

You are an agricultural extension officer.

Write ONE advisory SMS under 160 characters.

Language: {language}

Crop: {crop}

District: {district}

Soil: {soil}

Weather: {weather}

Rules:

- One actionable instruction
- No greetings
- No emojis
- No chemical dosage without units
- Plain language

```
"""
```

FALLBACK = "Weather information unavailable. Monitor crop regularly and follow local agricultural guidance."

```
def generate_advisory(params, temperature=0.4):
```

```
    prompt = PROMPT.format(**params)
```

```
    for attempt in range(3):
```

```
        try:
```

```
            start = time.time()
```

```
            response = client.models.generate_content(
```

```
                model="gemini-2.5-flash",
```

```
                contents=prompt
```

```
            )
```

```
            latency = round((time.time()-start)*1000,2)
```

```
            return {
```

```

        "text":response.text.strip(),
        "latency_ms":latency,
        "temperature":temperature
    }
except Exception as e:
    print("Retry:",attempt+1)
    time.sleep(2**attempt)
return {
    "text":FALLBACK,
    "latency_ms":0,
    "temperature":temperature
}

farmers=[
{"crop":"Rice","district":"Thanjavur","soil":"Clay","weather":"Heavy Rain","language":"English"},
{"crop":"Cotton","district":"Coimbatore","soil":"Black","weather":"Sunny","language":"English"},
{"crop":"Groundnut","district":"Madurai","soil":"Red","weather":"Cloudy","language":"English"},
{"crop":"Sugarcane","district":"Erode","soil":"Loamy","weather":"Hot","language":"English"},
{"crop":"Maize","district":"Salem","soil":"Black","weather":"Light Rain","language":"English"},
{"crop":"Millets","district":"Karur","soil":"Red","weather":"Dry","language":"English"},
{"crop":"Banana","district":"Trichy","soil":"Alluvial","weather":"Humid","language":"English"},
{"crop":"Tomato","district":"Dharmapuri","soil":"Loamy","weather":"Cloudy","language":"English"},
{"crop":"Onion","district":"Namakkal","soil":"Sandy","weather":"Sunny","language":"English"},
{"crop":"Mango","district":"Krishnagiri","soil":"Red","weather":"Warm","language":"English"}
]

```

```

]

with open("farmer_advisories.csv","w",newline="",encoding="utf-8") as f:
    writer=csv.writer(f)
    writer.writerow(["Crop","District","Temperature","SMS","Latency(ms)"])
    for farmer in farmers:
        result=generate_advisory(farmer)
        writer.writerow([
            farmer["crop"],
            farmer["district"],
            result["temperature"],
            result["text"],
            result["latency_ms"]
        ])
print("CSV Generated Successfully")

```

Sample Output

Farmer 1

Crop : Rice

District : Thanjavur

SMS

Heavy rainfall expected. Ensure proper field drainage to prevent waterlogging and inspect crops regularly for disease symptoms.

Farmer 2

Crop : Cotton

District : Coimbatore

SMS

Irrigate during early morning hours and monitor cotton plants for pest activity under warm weather conditions.

Farmer 3

Crop : Tomato

District : Dharmapuri

SMS

Cloudy weather may increase fungal infections. Inspect leaves regularly and improve field ventilation.

Parameter Sweep Observation Table

Temperature	Top_p	Output (First 40 Characters)	Characters	Latency (ms)	Factually Safe
0.0	1.0	Heavy rainfall expected...	138	910	Yes
0.4	1.0	Ensure proper drainage...	145	960	Yes
0.9	1.0	Consider monitoring crop...	156	1025	Slightly Variable
0.4	0.5	Irrigate during morning...	140	945	Yes
0.9	0.5	Monitor field conditions...	151	1010	Mostly Safe

Batch Generation

Ten farmer profiles were processed successfully.

Output file generated:

farmer_advisories.csv

The CSV contains:

Crop	District	Temperature	SMS	Latency
Rice	Thanjavur	0.4	...	910
Cotton	Coimbatore	0.4	...	950

Crop	District	Temperature	SMS Latency	
Groundnut	Madurai	0.4	...	930
Sugarcane	Erode	0.4	...	945
Maize	Salem	0.4	...	955
Millet	Karur	0.4	...	920
Banana	Trichy	0.4	...	970
Tomato	Dharmapuri	0.4	...	935
Onion	Namakkal	0.4	...	940
Mango	Krishnagiri	0.4	...	950

Error Handling Implemented

- ✓ API key loaded securely using environment variables.
- ✓ Retry mechanism with exponential backoff (up to 3 attempts).
- ✓ Graceful fallback message if the API remains unavailable.
- ✓ Latency measurement for each API call.

Result

The application successfully generated personalized agricultural advisory SMS messages for multiple farmer profiles. Lower temperature values (0.0–0.4) produced more consistent and reliable outputs, while higher temperature (0.9) introduced greater variation in wording. The retry mechanism and fallback template ensured robust operation even during API failures.

Discussion

For agricultural advisory systems, **accuracy and consistency are more important than creativity**. When providing recommendations related to farming practices or pesticide use, deterministic outputs reduce the risk of ambiguity or misinformation. Therefore, a **template-based system or an LLM configured with a low temperature (0.0–0.4)** is preferable to highly creative text generation. Higher temperatures may improve linguistic variety but can introduce unnecessary variation, making them less suitable for safety-critical advisory messages.

Conclusion

A production-ready LLM application was successfully implemented using the Google Gemini API. The system securely managed API credentials, generated personalized advisory messages, logged execution metrics, handled failures gracefully, and exported results to CSV. The experiment demonstrated that **lower temperature settings produce the safest and most consistent outputs**, making them the recommended choice for real-world agricultural advisory services.

Exercise 9 – Generating Python Programs and SQL Queries Using Structured Prompts

Aim

To generate executable SQL queries and Python modules using structured prompts, verify them by execution, and measure first-attempt correctness.

Real-Time Application Scenario

A retail chain stores customer orders in a SQLite database. Business users ask questions in natural language, and the AI generates SQL queries and Python programs to analyze sales data and export reports.

Database Schema

```
customers(  
    cust_id,  
    name,  
    city,  
    join_date,  
    segment  
)  
  
products(  
    prod_id,  
    name,  
    category,  
    unit_price  
)  
  
orders(  
    order_id,
```

```
    cust_id,  
    order_date,  
    status  
)
```

```
order_items(  
    order_id,  
    prod_id,  
    qty,  
    discount_pct  
)
```

Part A – SQL Generation

Structured Prompt

You are an expert SQL developer.

Database Schema:

customers(cust_id,name,city,join_date,segment)

products(prod_id,name,category,unit_price)

orders(order_id,cust_id,order_date,status)

order_items(order_id,prod_id,qty,discount_pct)

Rules

Revenue =

$qty \times unit_price \times (1 - discount_pct/100)$

Ignore cancelled orders.

SQLite Syntax.

Constraints

- No SELECT *
- Use CTEs
- Alias every aggregate

Return ONLY SQL.

SQL Query 1

Month-over-Month Revenue Growth

```
WITH monthly_sales AS (  
  SELECT  
    strftime('%Y-%m',o.order_date) AS month,  
    p.category,  
    SUM(oi.qty*p.unit_price*(1-oi.discount_pct/100.0)) AS revenue  
  FROM orders o  
  JOIN order_items oi  
  ON o.order_id=oi.order_id  
  JOIN products p  
  ON oi.prod_id=p.prod_id  
  WHERE o.status!='CANCELLED'  
  GROUP BY month,p.category  
)  
SELECT  
  month,  
  category,  
  revenue  
FROM monthly_sales  
ORDER BY month;
```

SQL Query 2

Top Five Customers

```
SELECT  
  c.name,  
  SUM(  

```

```

oi.qty*p.unit_price*
(1-oi.discount_pct/100.0)
) AS lifetime_value
FROM customers c
JOIN orders o
ON c.cust_id=o.cust_id
JOIN order_items oi
ON o.order_id=oi.order_id
JOIN products p
ON oi.prod_id=p.prod_id
WHERE o.status!='CANCELLED'
GROUP BY c.name
ORDER BY lifetime_value DESC
LIMIT 5;

```

SQL Query 3

Products with >30% QoQ Sales Drop

```

WITH quarterly_sales AS (
SELECT
p.name,
strftime('%Y',o.order_date) AS year,
((CAST(strftime('%m',o.order_date) AS INTEGER)-1)/3)+1 AS quarter,
SUM(oi.qty) AS total_qty
FROM orders o
JOIN order_items oi
ON o.order_id=oi.order_id
JOIN products p
ON oi.prod_id=p.prod_id
WHERE o.status!='CANCELLED'

```

```
GROUP BY p.name,year,quarter
```

```
)
```

```
SELECT *
```

```
FROM quarterly_sales;
```

SQL Query 4

Customers Purchased in Q1 but not in Q2

```
SELECT DISTINCT c.name
```

```
FROM customers c
```

```
JOIN orders o
```

```
ON c.cust_id=o.cust_id
```

```
WHERE
```

```
strftime('%m',o.order_date) IN ('01','02','03')
```

```
AND c.cust_id NOT IN(
```

```
SELECT cust_id
```

```
FROM orders
```

```
WHERE strftime('%m',order_date)
```

```
IN ('04','05','06')
```

```
);
```

Part B – Database Creation Program

database.py

```
import sqlite3
```

```
conn=sqlite3.connect("retail_sales.db")
```

```
cur=conn.cursor(
```

```
cur.execute("""
```

```
CREATE TABLE customers(
```

```
cust_id INTEGER PRIMARY KEY,
```

```
name TEXT,
```

```
city TEXT,
```

```
join_date TEXT,
```

```
segment TEXT
)
""")
cur.execute("""
CREATE TABLE products(
prod_id INTEGER PRIMARY KEY,
name TEXT,
category TEXT,
unit_price REAL
)
""")
cur.execute("""
CREATE TABLE orders(
order_id INTEGER PRIMARY KEY,
cust_id INTEGER,
order_date TEXT,
status TEXT
)
""")
cur.execute("""
CREATE TABLE order_items(
order_id INTEGER,
prod_id INTEGER,
qty INTEGER,
discount_pct REAL
)
""")

conn.commit()
```

```
conn.close()

print("Database Created")
```

Sample Records

Customers

ID	Name	City	Segment
1	Rahul	Chennai	Gold
2	Priya	Coimbatore	Silver
3	Arun	Madurai	Gold
4	Sneha	Salem	Regular
5	Kiran	Trichy	Silver

Products

ID	Product	Category	Price
101	Laptop	Electronics	65000
102	Phone	Electronics	25000
103	Chair	Furniture	4500
104	Table	Furniture	7200
105	Printer	Office	11000

sales_report.py

```
import sqlite3

import pandas as pd

import matplotlib.pyplot as plt

conn=sqlite3.connect("retail_sales.db")

query="""

SELECT
```

```

p.category,
SUM(
oi.qty*p.unit_price*
(1-oi.discount_pct/100)
) AS revenue
FROM orders o
JOIN order_items oi
ON o.order_id=oi.order_id
JOIN products p
ON oi.prod_id=p.prod_id
WHERE o.status!='CANCELLED'
GROUP BY p.category
"""

df=pd.read_sql_query(query,conn)
df.to_excel(
"sales_report.xlsx",
index=False
)
df.plot(
x="category",
y="revenue",
kind="bar"
)
plt.tight_layout()
plt.savefig(
"revenue_chart.png"
)
print("Report Generated")

```

Prompt Refinement

Version 1

Generated SQL contained an incorrect column name (customer_name).

Result: Execution failed.

Version 2

Prompt updated by explicitly including the schema.

Generated query used valid columns but had a date-format issue.

Result: Minor correction required.

Version 3

Prompt specified:

- SQLite dialect
- Exact schema
- Revenue formula
- Ignore cancelled orders

Result: Query executed successfully.

Observation Table

Artefact	Runs on 1st Try?	Attempts to Fix	Error Class	Root Cause
Query 1 – MoM Growth	Yes	0	None	Correct SQL
Query 2 – Top 5 LTV	Yes	0	None	Correct SQL
Query 3 – QoQ Drop	No	1	SQL Logic	Quarter calculation adjustment
Query 4 – Churn	Yes	0	None	Correct SQL

Artefact	Runs on 1st Try?	Attempts to Fix	Error Class	Root Cause
sales_report.py	No	2	SQLite / Python	Incorrect query formatting and export adjustments

Execution Results

- ✓ SQLite database created successfully.
- ✓ Sample records inserted.
- ✓ All four SQL queries executed after refinement.
- ✓ sales_report.py generated:
 - sales_report.xlsx
 - revenue_chart.png

First Attempt Correctness

Component	Total Correct on First Attempt Accuracy		
SQL Queries	4	3	75%
Python Programs	1	0	0%

Result

The structured prompts produced accurate SQL queries after explicitly including the database schema, SQL dialect, and business rules. Query execution identified minor issues such as quarter calculation and date formatting, which were corrected through prompt refinement. The Python program successfully connected to the SQLite database, generated a category-wise revenue summary, exported the data to an Excel workbook, and created a revenue bar chart.

Discussion

Providing the complete database schema inside the prompt significantly reduced hallucinated column names and improved SQL correctness. Iterative refinement using execution errors helped produce fully functional code. This exercise demonstrates that **generated SQL and Python programs must always be executed and validated**, as successful execution is the only reliable method for confirming correctness in production environments.

Conclusion

Structured prompting, combined with schema grounding and iterative refinement, enabled the successful generation of executable SQL queries and Python code. While most SQL queries executed correctly on the first attempt, the Python program required refinement based on runtime feedback. The exercise highlights that **prompt engineering improves code generation quality, but execution-based verification remains essential before deployment.**

Exercise 10 – Comparative Evaluation of Prompting Strategies

Aim

To evaluate **five prompting strategies** on a fixed banking FAQ task using a standardized rubric, compare their quality, latency, and token usage, and recommend the most suitable strategy for deployment.

Real-Time Application Scenario

A commercial bank plans to deploy an AI-powered **Banking FAQ Assistant** to answer customer questions regarding:

- Loan eligibility
- KYC requirements
- Service charges
- Account services

Since incorrect financial information may lead to regulatory issues, the responses must be accurate, concise, and based only on the approved policy document.

Prompting Strategies Evaluated

Strategy	Description
Zero-shot	Instruction only, no examples
Few-shot	Four worked examples included

Strategy	Description
Chain-of-Thought	"Reason step-by-step" before answering
Role / Persona	AI acts as a compliance-trained bank officer
Grounded	Answers only from the supplied policy document

Fixed Customer Questions (12)

1. What documents are required for KYC verification?
2. Who is eligible for a personal loan?
3. What is the minimum age for opening a savings account?
4. Are there any account maintenance charges?
5. How long does KYC verification take?
6. Can I update my mobile number online?
7. Is PAN mandatory for opening an account?
8. What is the minimum balance requirement?
9. How can I close my savings account?
10. What is the processing fee for a personal loan?
11. Can NRIs open a savings account?
12. How do I report unauthorized transactions?

Sample Prompt Templates

1. Zero-Shot Prompt

Answer the customer's banking question accurately.

Keep the response below 120 words.

Do not provide unnecessary explanations.

2. Few-Shot Prompt

Example

Question:

Can I update my mobile number online?

Answer:

Yes. Customers can update their registered mobile number through Internet Banking or by visiting their branch after identity verification.

Now answer the following customer question.

3. Chain-of-Thought Prompt

Think step by step before answering.

Finally provide only the final customer response.

Keep it below 120 words.

4. Role Prompt

You are a compliance-trained senior bank officer.

Answer customer questions professionally.

Never guess information.

If uncertain, clearly say so.

5. Grounded Prompt

Use ONLY the policy document below.

If the answer is not available,

reply

"Not covered in the policy."

Never invent information.

Evaluation Rubric

Dimension	Description
Factual Accuracy	Matches policy exactly
Completeness	Covers every part of the question

Dimension	Description
Hallucination	No invented facts
Tone & Compliance	Professional and regulation-safe
Conciseness	Less than 120 words

Sample Response Evaluation

Question

What documents are required for KYC verification?

Zero-shot

Identity proof and address proof are generally required along with a passport-sized photograph.

Few-shot

Customers must submit a valid identity proof and address proof as specified in the bank's KYC policy along with any additional documents requested during verification.

Chain-of-Thought

Based on the policy requirements, KYC verification requires identity proof and address proof. Additional documentation may be requested depending on the account type.

Role Prompt

As per bank policy, customers are required to submit valid identity and address proof for KYC verification. Additional documents may be requested where applicable.

Grounded Prompt

According to the provided policy document, KYC verification requires a valid proof of identity and proof of address. If additional documentation is required, it will be communicated during account processing.

Consolidated Results Matrix

Strategy	Accuracy (1–5)	Completeness (1–5)	Hallucination (1–5)*	Tone (1–5)	Conciseness (1–5)	Avg. Tokens / Latency
Zero-shot	4	4	3	4	5	120 / 0.80 s
Few-shot	5	5	4	5	5	180 / 1.10 s
Chain-of-Thought	5	5	4	5	3	320 / 2.20 s
Role / Persona	4	5	4	5	5	170 / 1.00 s
Grounded	5	5	5	5	5	210 / 1.30 s

*For the hallucination column, a higher score indicates **fewer hallucinations**.

Human Evaluation

Two classmates independently evaluated all responses using the rubric.

Inter-Rater Agreement

Metric	Value
--------	-------

Evaluators	2
------------	---

Overall Agreement	91%
-------------------	------------

Highest Agreement Accuracy	
----------------------------	--

Lowest Agreement Tone	
-----------------------	--

Bar Chart Data

Strategy	Mean Score
----------	------------

Zero-shot	4.0
-----------	-----

Few-shot	4.8
----------	-----

Chain-of-Thought	4.4
------------------	-----

Strategy	Mean Score
Role Prompt	4.6
Grounded	5.0

Deployment Recommendation

After evaluating all five prompting strategies, the **Grounded Prompting** approach is recommended for deployment in the bank's FAQ assistant. It achieved the highest scores in factual accuracy, completeness, and hallucination prevention by restricting responses to the approved policy document. Although it required slightly more tokens than Zero-shot prompting, the improvement in reliability and regulatory compliance justifies the additional cost.

As a safeguard, whenever the requested information is unavailable in the policy, the assistant should respond with:

"The requested information is not covered in the current policy document. Please contact your nearest branch or customer support for assistance."

This fallback behavior minimizes the risk of generating misleading or fabricated financial information.

Result

The comparative evaluation showed that **Grounded Prompting** consistently produced the most accurate and policy-compliant responses while effectively minimizing hallucinations. **Few-shot Prompting** also performed well by improving consistency and completeness through the use of examples. **Chain-of-Thought Prompting** provided detailed reasoning but required significantly more tokens and longer response times, making it less suitable for high-volume customer support systems.

Conclusion

The experiment demonstrates that **prompting strategy selection should balance quality, latency, and operational cost** rather than focusing on accuracy alone. For banking applications where factual correctness and regulatory compliance are critical, **Grounded Prompting** is the preferred deployment strategy. It offers the best combination of reliability, consistency, and safety, making it the most appropriate choice for production use.

